Parallelising Timed-Automata Reachability:
A Shared-Memory (OpenMP) and a GPU (Metal/CUDA) Backend for a Timed-CTL Model Checker

PTCTLMC Project

July 20, 2026

Abstract

We describe two parallel backends for the reachability engine of PTCTLMC, a model checker for timed automata and networks thereof. The engine explores a *zone graph*: symbolic states pair a discrete location/variable vector with a convex *zone* of clock valuations, represented as a Difference Bound Matrix (DBM). Two operations dominate the cost: computing successor zones, and merging them into a *passed list* under abstraction-based subsumption and exact convex union. Both are embarrassingly parallel *across* symbolic states but stubbornly sequential *within* a merge bucket. Our first backend shards the passed list by a packed discrete key and lets OpenMP workers expand the frontier and merge disjoint shards without locks. Our second backend runs the entire level loop on an Apple Metal GPU: one GPU thread per successor "recipe" and one GPU thread per discrete-state bucket, with a host-side marshaling layer that we ultimately parallelise across the CPU cores as well—so the machine computes with the GPU *and* all cores at once. We validate that every backend explores the *identical* zone graph as UPPAAL—matching its stored- and explored-state counts *integer-for-integer* on Fischer, two-lock and arbiter—so the speed-ups compare like with like. Across an evaluation whose per-model size grid is capped by the machine's memory ceiling, both parallel backends beat single-threaded `verifyta` on every model (OpenMP by up to 3.5×). The Metal backend additionally beats the 12-thread OpenMP backend on the *expansion-dominated* models—Fischer, two-lock and arbiter—reaching 3.3–6.2× over `verifyta`; a single optimisation that stops the host copying freshly generated zones (the merge kernel reads them in place from the expand kernel's output) is what turns the largest Fischer instance from a loss into a win. The one place the naive GPU merge *loses*—a *merge-dominated* model (dining philosophers), where its one-thread-per-bucket kernel is the wrong shape and collapses to 122 s—we *fix* with an `-adaptive` mode that calibrates the GPU-versus-12-core-CPU merge throughput per level and routes each level to the faster path: a 14–47× speed-up that makes the adaptive Metal backend the fastest configuration on *every* model, beating OpenMP throughout and `verifyta` by 3.4–7.3×. Its cost is memory—the GPU backends carry a double-buffering premium—so peak RAM, not run time, sets the ceiling on a 24 GB laptop. We give the design, intuition and diagrams for both backends, an honest account of three optimisations that *failed*, and a CUDA port that inherits the adaptive merge for datacentre GPUs.

1 Introduction

A *timed automaton* is a finite automaton equipped with real-valued clocks that advance synchronously and can be compared against integer constants and reset on transitions. Verifying a property such as *"two processes are never simultaneously in their critical section"* reduces to a reachability question over an infinite state space (locations × clock valuations in $\mathbb{R}^n_{\geq 0}$). The classical solution replaces sets of clock valuations by *zones*—convex polyhedra defined by difference constraints $x_i - x_j \bowtie c$—and explores the resulting *zone graph*, which can be kept finite by sound abstractions. Real benchmarks (Fischer mutual exclusion, CSMA/CD, train–gate) are *networks* of such automata communicating through shared bounded integer variables and synchronisation channels; their product state spaces explode combinatorially, and this is exactly where wall-clock time is spent.

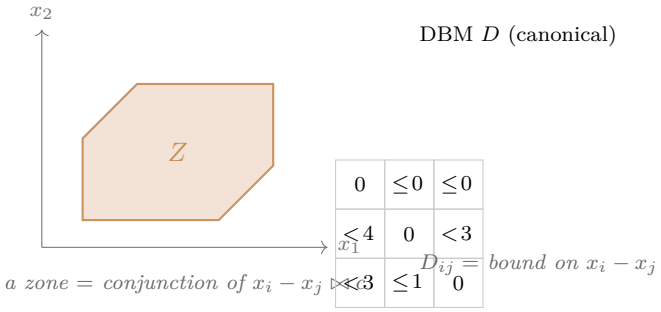


Figure 1: A clock zone and its DBM. All backends manipulate the DBM with the *same* `__host__ __device__` operations, so GPU code runs CPU-validated algebra.

PTCTLMC is a from-scratch checker whose reference (sequential) engine is validated against UPPAAL. This report concerns *how we made it fast in parallel*, on two very different pieces of hardware:

- a **shared-memory** backend using **OpenMP** across CPU cores (section 3); and
- a **GPU** backend using Apple **Metal** (section 4), mirrored to **CUDA** for datacentre GPUs (section 5).

Both share one control structure—a level-synchronised breadth-first search—and the same correctness oracle. We foreground *intuition*: what is being computed, where the parallelism comes from, and why a natural-looking optimisation can make things slower. All diagrams are TikZ-drawn and every performance number is reproducible with the benchmark generators shipped in `benchmarks/`.

2 Background: the zone graph and where time goes

A *symbolic state* of a network is a pair $s = (\ell, Z)$ where $\ell = (\ell_1, \ldots, \ell_p, v_1, \ldots, v_m)$ is the *discrete part* (one location per process plus the shared integer variables) and Z is a *zone* over the n shared clocks. A zone is stored as a *Difference Bound Matrix* (DBM): an $(n{+}1) \times (n{+}1)$ matrix D whose entry D_{ij} is the tightest bound on $x_i - x_j$ (index 0 is a fixed zero clock), each bound encoded as a single machine integer $(\text{value} \ll 1) \,|\, \text{strict}$. Every zone operation—intersection with a guard, clock reset, time-elapse ("delay"), emptiness—is a handful of nested loops over D and runs identically on CPU and GPU (fig. 1).

The level-synchronised loop. Reachability is a breadth-first exploration of the zone graph organised in *levels* (fig. 2). Starting from the initial symbolic state, each iteration (i) *expands* the current frontier—applies every enabled transition to every frontier state to produce successor zones—and (ii) *merges* those successors into a *passed list* keyed by discrete state, keeping only an *antichain* of maximal zones per key. Survivors of the merge form the next frontier. The property holds/fails as soon as a target state is generated or the frontier empties.

The two hot operations. *Expansion* is uniform, data-parallel arithmetic: one independent DBM pipeline per (frontier-state, transition). *Merging* is where subtlety lives. For each discrete key we maintain an antichain under the *aLU preorder* $\preceq_{\text{aLU}}$—the abstraction-based simulation of Herbreteau–Srivathsan–Walukiewicz that is strictly coarser than zone inclusion and so prunes far more—plus an *exact convex union*: if two stored zones' union happens to be convex, they are fused into one, which is what collapses the stored-state count to match UPPAAL. Both tests are $O(\dim^2)$–$O(\dim^3)$ DBM comparisons; a discrete key with k stored zones costs $O(k^2)$ of them. The antichain per key is small, but there are millions of keys.

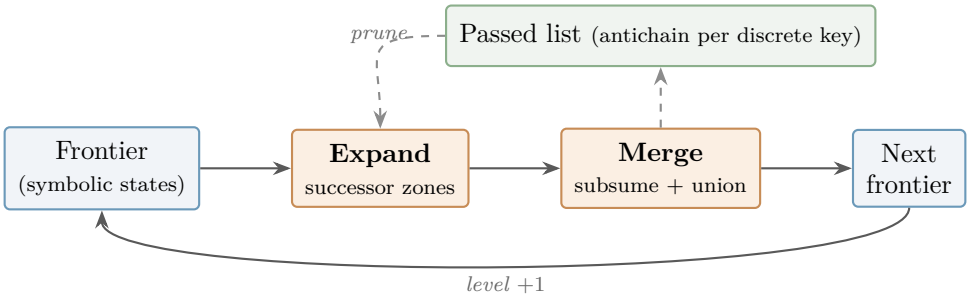


Figure 2: One iteration of the level-synchronised BFS. *Expand* and *merge* dominate the runtime; both parallelise across states, and are the targets of sections 3 and 4.

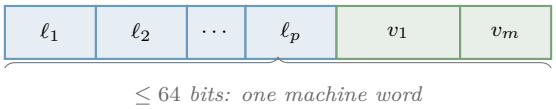


Figure 3: The packed discrete key *dk*: locations (blue) and bounded integers (green) laid out in disjoint bit-fields. Hashing/comparing a symbolic state's discrete part is now one word op.

The discrete key. To index the passed list we pack the whole discrete part into a single 64-bit integer: a fixed bit-field per process location and per integer variable (fig. 3). This mirrors UPPAAL's "compressed integers" and turns key comparison and hashing into single-word operations—essential once keys are compared billions of times. The field widths come from the declared ranges (`int[lo,hi]`); getting them wrong silently collides distinct states (a real bug we found and fixed).

3 The OpenMP backend

The shared-memory backend keeps the sequential control flow and parallelises the two hot phases across cores. The design principle is *ownership without locks*: partition the work so that no two threads ever touch the same passed-list bucket.

Sharding by discrete key. We split the passed list into S shards (one per worker) by hashing the discrete key: $\text{shard}(dk) = (dk \cdot \phi) \bmod S$. Because a given discrete state always lands in the same shard, each shard is a *self-contained* antichain owned by exactly one thread during the merge—so the $O(k^2\ \text{dim}^3)$ convex-merge runs fully in parallel with no synchronisation (fig. 4).

Allocator contention is the real enemy. A subtle lesson: with the merge correctly parallelised, scaling still collapsed at large N (from $\sim$3.6$\times$ to $\sim$1.8$\times$ on 12 cores). Profiling showed the culprit was not the algorithm but `malloc`: every churned zone is a `malloc`/`free` pair, and `malloc` holds a *process-global* lock, so all workers serialised on the allocator. Replacing zone storage with a *per-thread free-list pool* (recycling equally-sized DBM buffers without touching the global heap) restored scaling. The message generalises to the GPU: *the bottleneck is rarely the compute; it is the memory plumbing around it.*

Algorithm 1 summarises the level loop; each **parallel-for** is a barrier, and the routing step in between is cheap serial bookkeeping.

4 The Metal GPU backend

GPUs excel at the uniform arithmetic of expansion, but the merge—branchy, data-dependent, antichain-shaped—is exactly what GPUs are *bad* at. The engineering question is therefore not

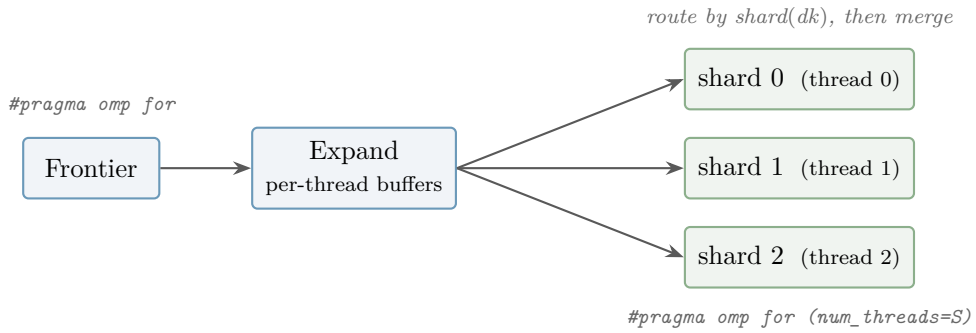


Figure 4: OpenMP parallelisation. Workers expand the frontier into private buffers, successors are routed to shards by a hash of the discrete key, and each shard—a disjoint slice of the passed list—is merged by a single owning thread. No locks are needed anywhere.

Algorithm 1 OpenMP level loop (one iteration)

parallel for $s \in$ frontier: ▷ expand tls[tid] += Successors(s)
for $s \in$ tls[$\cdot$] **do** ▷ route (serial, cheap)
  if IsTarget(s) **return** true
  byshard[shard(dk_s)] += (dk_s, s)
end for
parallel for $t \in [0, S)$: ▷ merge disjoint shards
  for (dk, s) $\in$ byshard[t]: AddTo(shard[t], dk, s)
  collect this level's survivors into next frontier

"can the DBM math run on the GPU" (it can) but "can we keep the GPU fed without drowning in host↔device traffic." This section develops the answer from the decomposition outwards: first *where* the parallelism actually is (section 4.1), then the data layout that exposes it (section 4.2), then the two kernels (sections 4.3 and 4.4), then the data path that feeds them (section 4.5), and finally the adaptive router that repairs the one case where the decomposition breaks down (section 4.7).

4.1 Where the parallelism is—and where it is not

Everything in this section follows from one observation about the level loop, so it is worth making precisely. The loop has two hot phases and they have *opposite* characters.

Expansion is unboundedly and uniformly parallel. For a frontier of F symbolic states, the successors of state s depend only on s and on the (read-only) network description. No two frontier states interact, and no two successors of different states share any mutable location. The available parallelism is therefore the whole Cartesian product {frontier states} × {enabled transitions}—at Fischer N=11 that is 1.44M independent tasks in a single level—and every task runs the *same* straight-line sequence of DBM operations on its own private zone. Uniform control flow, no synchronisation, no write conflicts: this is the textbook GPU workload.

The merge is parallel only across discrete keys. The successors must then be folded into the passed list, and here the tasks are *not* independent. Two successors that share a discrete key dk interact—each may subsume the other, or the two may be absorbed into a single zone by the exact convex union—while two successors with different keys can never interact at all, because the passed list is partitioned by dk. The merge is therefore *embarrassingly parallel across buckets and strictly sequential within one*. Two consequences follow immediately, and they run the rest of this paper:

1. The available parallelism is not the number of successors but the **number of distinct buckets touched this level**. A level that produces a million successors into a thousand buckets offers a thousand-way parallelism, not a million-way.
2. The per-task cost is **quadratic in the bucket's antichain size**: folding k zones into a bucket costs $O(k^2)$ subsumption/union tests of $O(\dim^2)$–$O(\dim^3)$ each. The work is thus distributed as the *square* of a quantity we do not control and cannot cheaply predict.

A one-thread-per-bucket mapping is the natural way to exploit (1). But (2) means that mapping is only efficient when buckets are *many and thin*; when they are *few and fat* a handful of weak GPU lanes carry the entire level while the rest of the device idles. Figure 7 draws both regimes side by side, section 6 shows they both occur in practice, and section 4.7 is our fix.

4.2 Data layout: flat arrays, no objects

The CPU engine stores a symbolic state as a small object graph (a location vector, an integer vector and a `Dbm`, each separately heap-allocated). That representation cannot cross to a GPU, and converting it per level was what made our first GPU attempt three times *slower* than OpenMP (section 4.5). The GPU path therefore abandons objects entirely and keeps every level in *flat, struct-of-arrays* form:

- the frontier is three parallel arrays—`fl` ($F\times p$ locations), `fi` ($F\times m$ integers), `fz` ($F\times \dim^2$ zone words);
- the passed list is `unordered_map<uint64, vector<raw_t»`: one contiguous run of zone words per discrete key, addressed by the packed key of fig. 3;
- recipes and merge metadata are CSR-style index arrays (offsets + payload), never pointer-linked structures.

Two properties matter. First, every array is a single allocation that the GPU can read directly, so "uploading" a level is at worst one `memcpy` and usually not even that (section 4.5). Second, because a zone is exactly $\dim^2$ machine words at a known offset, a kernel thread computes the address of *its* zone arithmetically—no indirection, fully coalesced access.

4.3 Expand: one GPU thread per successor recipe

A network transition can be a plain location step, a binary handshake ($a! \parallel a?$), or a broadcast ($a!$ with all enabled $a?$). Rather than write three kernels, we unify them into one abstraction: a *recipe* is a short list of participating $(\text{process}, \text{edge})$ pairs—one pair for a local step, two for a handshake, $1{+}K$ for a broadcast with K receivers. A recipe is exactly "one potential successor", so *one recipe becomes one GPU thread* and the kernel needs no case analysis over channel kinds: it simply loops over the participant list.

Why the host enumerates and the GPU computes. Deciding *which* recipes exist is irregular work: matching senders to receivers per channel, evaluating integer guards, honouring committed-location priority, and taking a cross product over receivers for broadcasts. It is branchy, its output size is data-dependent, and it is cheap—integer tests over a handful of arrays. Deciding *what each recipe produces* is the opposite: a fixed sequence of dense $O(\dim^2)$ matrix operations, identical for every recipe. So we split along that seam. The host emits the recipe list as CSR arrays (`ws`, `part_off`, `part_cnt`, `part_proc`, `part_edge`); the GPU runs the zone pipeline. Figure 5 shows the whole path.

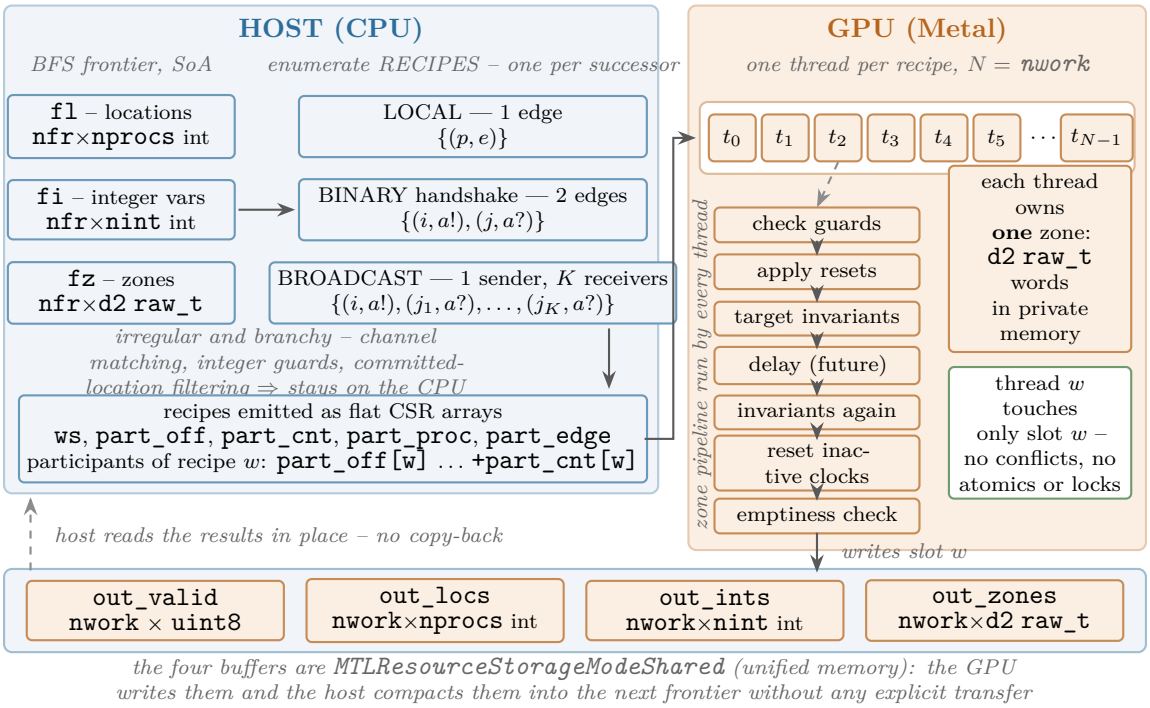


Figure 5: Expansion, end to end. The host holds the frontier as three flat struct-of-arrays and enumerates *recipes*—the irregular, branchy part (channel matching, integer guards, committed-location priority)—emitting them as CSR index arrays. The GPU then runs *one thread per recipe*, each executing the identical zone pipeline on its own private zone and writing only its own output slot, so the kernel needs no atomics and no synchronisation. All four output buffers are unified memory, so the host reads them in place with no copy-back.

What one thread does. Thread w reads its recipe's participant list, copies its source zone into thread-local storage, and runs exactly the pipeline of section 2: conjoin every participant's clock guard, apply every reset, conjoin the combined target invariant, let time elapse unless some participant is urgent or committed, re-apply the invariant, reset the clocks no process can still read, and test emptiness. It then writes its verdict and its zone to slot w of the output arrays. Because slot w is private to thread w, there is no synchronisation anywhere in the kernel and no atomics—the invalid slots are simply skipped by the host afterwards. The cost is register pressure: each thread holds a dim^2 zone (plus scratch), which is what caps the supported clock count at $\mathrm{dim} \leq 24$.

4.4 Merge: one GPU thread per discrete-state bucket

After expansion the host groups the valid successors by discrete key—integer hashing over the packed *dk*, cheap and cache-friendly—and builds a *work pool* describing the level's merge as flat arrays: for bucket b, `work_off[b]` is where its slice starts, `exist_cnt[b]` how many zones its existing antichain contributes, and `new_idx[new_off[b] ..]` lists the expand-output indices of its new zones. The kernel then assigns *one thread per bucket*.

That thread replicates the sequential `AddTo` *exactly*, in the same order, for each new zone in turn: (a) test whether the new zone is aLU-subsumed by any stored zone—if so, discard it; (b) drop any stored zones subsumed by the new one, compacting the slice and carrying the "is new" flags along; (c) repeatedly attempt the exact convex union, absorbing any stored zone whose union with the new one is itself convex; (d) append the survivor and mark it new. Preserving the order matters: the convex union is order-sensitive (section 2), so a different order would give a different—still sound, but different—stored count. Crucially the convex union runs on the GPU

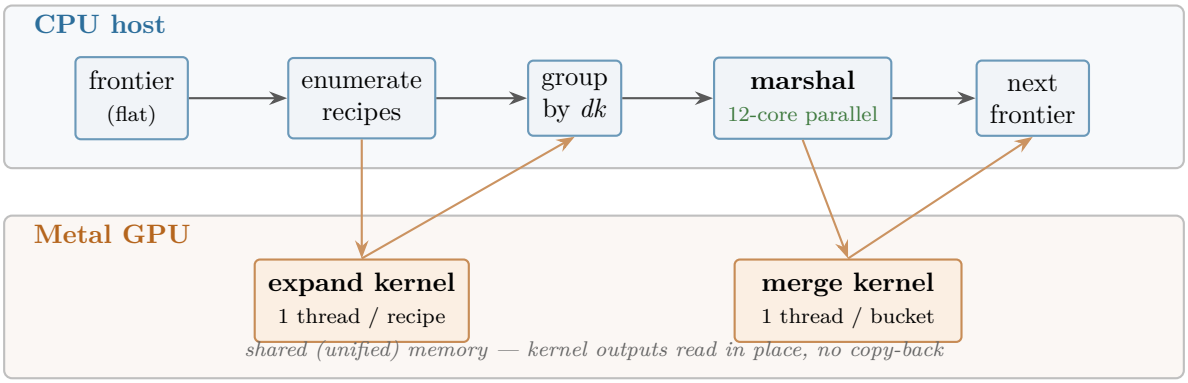


Figure 6: The Metal level loop. The GPU runs the two kernels (expand over recipes, merge over buckets); the host does only the irregular discrete work (enumeration, grouping) and the zone *marshalling*, which we parallelise across all CPU cores. Buffers live in unified memory, so kernel outputs are read *in place* rather than copied back—the machine uses the GPU and all CPU cores simultaneously.

too, with no shortcut, which is why the GPU backend's stored-state count matches UPPAAL's rather than merely approximating it.

The mapping is a good fit exactly when the level offers many thin buckets, and a poor one when it does not—the two regimes of fig. 7.

4.5 Keeping the GPU fed: the flat unified-memory data path

The first working version was three times *slower* than OpenMP, and the profiler explained why: the GPU kernels were ∼25% of the time, and the remaining ∼75% was the host converting between `SymState` objects and flat arrays, allocating hundreds of megabytes of Metal buffers *every level*, and copying each zone five or six times. We rebuilt the data path around four ideas, each validated by re-profiling:

1. **Flat, resident state.** No per-state objects; the frontier and passed list are flat `raw_t` arrays. This removed a per-successor heap allocation and a map rebuilt from `Dbm` copies every level.
2. **Persistent buffers, written in place.** Apple GPUs share memory with the CPU (*unified memory*). We keep grow-on-demand `MTLBuffer`s and *gather directly into their contents*, so there is no separate "upload"; symmetrically, kernel outputs are read straight from the buffer (`const raw_t*&` out-parameters), with no read-back copy.
3. **Parallel host marshalling.** The remaining per-bucket gather/rebuild is over independent buckets, so it runs under `#pragma omp parallel for`. The metal backend is thus *GPU + 12 cores*, versus OpenMP's 12 cores—strictly more hardware in flight.
4. **New-zone gather elimination.** Even after the above, the host still touched every freshly generated ("new") zone *twice*: once to stage it into the merge's work buffer, once to read the merged result back. We removed the first pass entirely: the merge kernel reads new zones *in place* from the expand kernel's output buffer, indexed by a `new_idx` array, so the host stages only each key's *existing* antichain. This halved the host memcpy profile and, on Fischer N=12, turned a ∼17.6 s *loss* to OpenMP into a ∼12.2 s *win*—the single change that makes Metal beat the CPU across the expansion-dominated models (section 6).

A final micro-optimisation: a bucket all of whose new zones are subsumed is *unchanged*, so we skip its rebuild and frontier gather entirely.

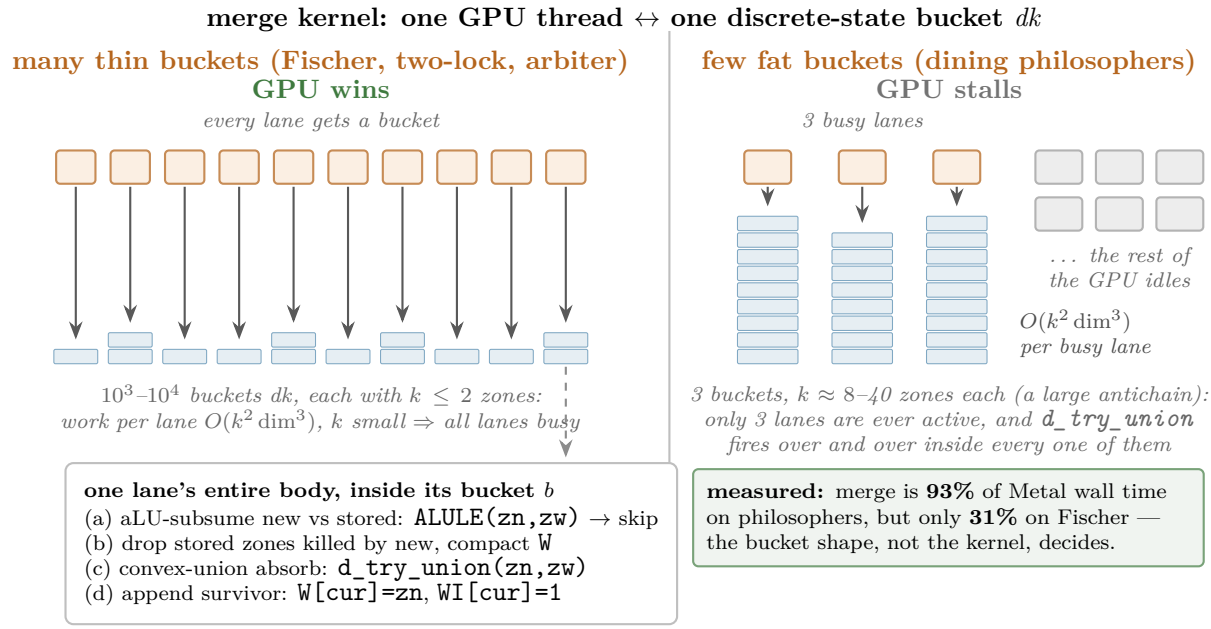


Figure 7: Why the merge kernel's *shape* decides everything. One GPU thread is assigned per discrete-state bucket. **Left:** many thin buckets (Fischer, two-lock, arbiter)—every lane gets a bucket with a tiny antichain, all lanes stay busy, and the GPU wins outright. **Right:** few fat buckets (dining philosophers)—a handful of lanes each grind through an $O(k^2 \dim^3)$ antichain while the rest of the device idles. The inset lists what one lane actually executes. This single structural difference is the whole story of section 6: measured, the merge is 93% of Metal's wall time on philosophers but only 31% on Fischer.

Why "GPU + 12 cores" is literal. Point (3) deserves unpacking, because it is what makes the Metal backend faster than the OpenMP backend rather than merely a different way of being as fast. A level is not "GPU work" followed by "CPU work"; it is a pipeline in which each phase runs on the processor that suits it. The GPU executes the two kernels; the CPU, on all twelve cores simultaneously, does the work the GPU is bad at—enumerating recipes, hashing successors into buckets, gathering each bucket's existing antichain, and rebuilding the passed list and the next frontier. Those host phases are themselves data-parallel over independent buckets, so they run under `#pragma omp parallel for`, and the per-thread zone pool of section 3 keeps them off the global allocator. The OpenMP backend has twelve cores; the Metal backend has twelve cores *and* a GPU, and neither sits idle for a whole phase. The one subtlety is that the parallel rebuild writes into an `unordered_map` while other threads read it, which is safe only because element pointers are stable across insertions—so we pre-create the nodes serially and fill their values in parallel.

Why the copies were the enemy at all. It is worth stating the underlying economics, since it explains both the wins here and the failures in section 4.6. A zone is $\dim^2$ words—676 bytes at 12 clocks—and a large level moves millions of them. At Fischer N=11 the expand output alone is 1.44M zones (0.77 GB) and the merge work pool another 1.21M (0.65 GB). Every *additional* pass over that data costs more than the arithmetic the pass exists to serve: the DBM operations are $O(\dim^2)$ on data that takes $O(\dim^2)$ words to move, so the kernel is bandwidth- rather than compute-bound. That is why removing a single copy (point 4) is worth more than any amount of kernel tuning, and why two later attempts to remove *different* copies backfired the moment they replaced a large sequential transfer with a scattered or serialised one.

```
// bucket b's final antichain, straight from the GPU buffer:
for (int i = 0; i < out_count[b]; ++i) {
  const raw_t* z = out_zones + (work_off[b]+i)*d2;
```

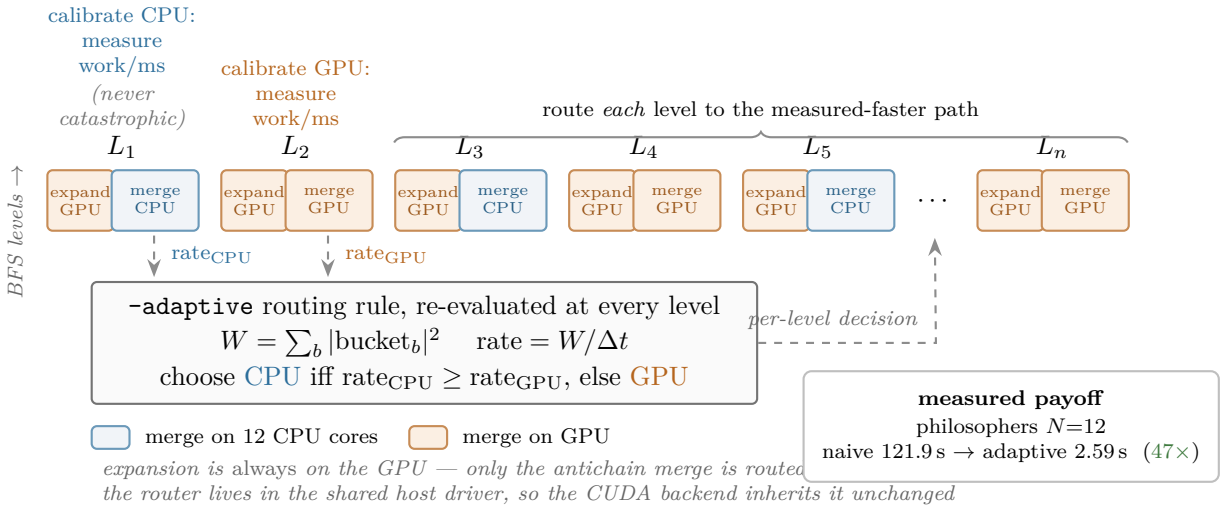


Figure 8: Adaptive merge routing. Expansion always runs on the GPU; only the antichain merge is routed. The driver *measures* instead of guessing: it calibrates the CPU path first (never catastrophic), then the GPU path, recording each one's throughput $W/\Delta t$ over the work proxy $W = \sum_b |\text{bucket}_b|^2$, and thereafter sends every level to whichever processor is measured faster. Because the router lives in the shared host driver rather than in device code, the CUDA backend inherits it unchanged.

```
  if (out_isnew[work_off[b]+i]) /* -> next frontier */;
}
```

Listing 1: The merge output is read in place from unified memory—no copy.

4.6 Two data-path optimisations that failed (and why)

Intuition said the ultimate win was a *GPU-resident arena*: keep every passed-list zone on the GPU forever, store only `dk -> (offset,count)` on the host, and have the merge read existing zones *in place* from the arena—eliminating even the existing-zone gather. We implemented it fully and it was *exact-correct*, yet it *regressed* every problem size. Two reasons, both instructive: (i) the kernel must still copy each bucket's existing antichain into a fresh append region, and that copy now runs *single-threaded per bucket on the GPU* instead of across 12 CPU cores; and (ii) an append-only arena needs periodic compaction, a multi-gigabyte host `memmove`. A second attempt—writing each level's next frontier *directly* into a persistent GPU buffer instead of gathering it—also regressed (Fischer N=12: 12.2→14.0 s), because it turned one large *sequential* gather into many *scattered* host writes into GPU-shared memory, which are slow once the working set is large. We reverted both. The lesson is the recurring theme of this report: on a memory-bound workload, moving a copy onto slower hardware—or scattering it—loses even when it removes a copy.

4.7 Adaptive merge routing: CPU or GPU, chosen per level

The one-thread-per-bucket merge is the *right* shape only when a level has *many, thin* buckets: on Fischer, ∼70k buckets each hold a tiny antichain, so thousands of GPU lanes each do a little work. It is the *wrong* shape when the per-bucket work is heavy. On dining philosophers the exact convex union fires constantly, each of a few thousand buckets carries an $O(k^2 \dim^3)$ load, and a handful of weak GPU lanes grind through it while the CPU cores idle. The profiler is unambiguous: the merge kernel is ≈93% of the Metal wall-clock on philosophers versus ≈31% on Fischer. Tellingly, *static* bucket-size heuristics (bucket count, mean/max antichain size) do *not* separate the two

cases—the discriminator is the branchy convex-union cost, which those metrics cannot see. So the driver *measures* rather than guesses.

With the `-adaptive` flag, the flat driver keeps a host port of the merge kernel, `flat_merge_bucket`, that runs the identical aLU-subsumption and convex-union on the CPU using the very same `__host__ __device__ dbm_ops` routines, parallelised across buckets with `#pragma omp parallel for`—i.e. the merge on all 12 cores instead of one GPU thread each. For every BFS level it estimates the merge work as $\sum_b |\text{bucket}_b|^2$ and records each path's *throughput* (work per millisecond); it calibrates the CPU path first (never catastrophic), then the GPU path, then routes every subsequent level to whichever is *measured* faster, updating the estimate as it goes (fig. 8). *Expansion always stays on the GPU*—only the merge is routed. Because this lives in the *shared* flat driver and not in device code, the CUDA backend (section 5) inherits it unchanged.

The default stays the naive one-thread-per-bucket merge; `-adaptive` is opt-in. The effect is large where it matters and harmless elsewhere: on philosophers $N{=}11$ the Metal wall-clock drops from 8.1 s to 0.6 s—a 13× speed-up that turns the backend's *worst loss* into a win over both the 12-thread OpenMP backend (1.2 s) and `verifyta` (2.6 s)—while the already-merge-light Fischer still improves slightly (2.64 → 2.31 s). The pathology is GPU-specific: OpenMP's merge is *already* parallel across buckets (section 3), so it has nothing to route—an `omp` "adaptive" mode would toggle nothing.

5 The CUDA port

Because the *control* of the level loop lives in host code and only the *device* layer is platform-specific, porting to CUDA is a device-for-device exercise. What makes that true is a deliberate three-way split, and it is worth naming because it is the part of the design that generalises beyond this tool.

One driver, two devices. The abstract interface `NetGpuExpander` exposes exactly three operations—`expand`, `stage_merge`, `merge`—and the entire level loop (algorithm 1's GPU analogue: grouping by *dk*, prefix sums, the passed-list rebuild, the frontier assembly, and the adaptive router of section 4.7) lives *above* that interface in `netreach.cpp`. Neither backend contains any control flow. The practical payoff is concrete: the `-adaptive` router and the `-chunking` batcher were both implemented once, in the shared driver, and the CUDA backend acquired them without a line of device code changing.

Shared algebra, not re-implemented algebra. The zone operations in `dbm_ops.hpp` are annotated `__host__ __device__`, so a CUDA kernel calls the *same* functions the CPU unit tests validate—the port inherits correctness rather than re-deriving it. Only one routine needed a device twin: the exact convex union is host-only (it is `inline` and sizes its scratch to the CPU's `kDbmMaxDim`= 64), so the `.cu` file carries a `d_try_union` with device-sized scratch. Metal cannot share C++ headers at all—MSL is a separate language—so its kernels embed a transcription of the same operations, which is precisely why we audit the two engines against each other rather than trusting them independently (section 6).

Mapping the memory model. The Metal path leans on Apple's unified memory: buffers are `MTLResourceStorageModeShared`, so a buffer's `.contents` pointer is written by the host and read by the GPU with no copy. CUDA's counterpart is *managed* memory (`cudaMallocManaged`), which presents the same single-pointer illusion, and that correspondence is what lets the identical host driver write merge inputs and read kernel outputs through raw pointers on both platforms. The illusion is not free in the same way, though, and the difference matters for one feature: on Apple silicon the memory really is shared, whereas on a discrete GPU managed memory is migrated page-by-page on fault. The `-adaptive` router runs the per-bucket merge *on the host* over buffers the expand kernel just wrote on the *device*, which is free on a Mac and a migration storm on an

H100; the CUDA backend therefore marks those pools `cudaMemAdviseSetAccessedBy` so the runtime keeps them mapped for both processors. This is the one place where a shared-driver feature needed a device-specific accommodation, and it is exactly the kind of thing a device-for-device port hides until it is measured.

Datacentre GPUs (H100) have both far higher arithmetic throughput and much wider memory bandwidth—and thousands more lanes—which are precisely the resources that bound the laptop GPU at the two places it currently *loses* (section 6): the memory-bound top of the scaling curve, and the merge-heavy models where a wider machine would let a threadgroup-per-bucket merge parallelise the antichain scan instead of stalling one lane per bucket. We therefore expect the CUDA backend both to raise the ceiling and to soften the merge-dominated worst case.

6 Experimental evaluation

Setup. Apple M4 Pro (12 CPU cores; integrated GPU; 24 GB unified memory), macOS. We measure nine configurations: `seq` (one core), `omp` (12 threads), `metal` (GPU + 12 cores, *naive* one-thread-per-bucket merge), `metal-ad` (`metal -adaptive`, section 4.7), a sweep of `-chunking` at 4k/16k/64k/256k frontier states per batch (all on top of `-adaptive`), and the reference UPPAAL `verifyta` 5.0 (single-threaded). Wall-clock is a discarded warm-up then the *minimum* of k timed runs (k shrinks from 5 toward 1 as a single run grows past 15 s; deterministic baselines need no repeats). Peak RSS is captured with `/usr/bin/time -l` in the same probe. Because `metal-ad` allocates host merge buffers on top of the GPU buffers it is the memory-hungriest config, so the N-grid per model is capped to keep its peak RSS below ∼13 GB on the 24 GB machine; the full run is 28 configurations in `benchmarks/results_full.csv`. Five generated benchmarks exercise different characteristics: Fischer (shared integer), *two-lock* (two shared integers), *arbiter* (binary channels), *CSMA/CD* (broadcast), and *dining philosophers* (many binary channels; large per-key zone antichains). Each timed query is a *safety* property that *holds*, forcing exploration of the whole reachable state space (a satisfiable `E<>` query would stop at the first witness, measuring nothing).

Fairness: the state counts match UPPAAL exactly. A speed-up over a tool doing *more* work is meaningless, so the primary control is that both tools explore the *same* zone graph. PTCTLMC's *stored* and *explored* symbolic-state counts equal `verifyta`'s "States stored" / "States explored" *integer-for-integer* on Fischer, two-lock and arbiter at every size (table 1)—the same reachable space, the same abstraction, the same work. On CSMA/CD and philosophers PTCTLMC stores *fewer* states than UPPAAL (1.6–2.0×), because its exact convex-union merge (section 2) fuses more aggressively than UPPAAL's default representation; this is sound (verdicts agree) and is reported as a separate representational advantage rather than folded into the timing claim. Because that convex union is order-sensitive, a parallel schedule's *stored* count varies by a handful of states run-to-run (philosophers N=8: `seq` = 6992 stable, `metal` = 7006; a 0.2% spread) while the verdict is invariant; the deterministic `seq` backend is therefore the parity oracle, and the parallel backends are checked only for verdict agreement (all 28 configurations agree).

Results. Table 2 gives representative wall-clock times, fig. 9 the full speed-up-over-`verifyta` curves, and fig. 10 what `-adaptive` buys. Two facts hold everywhere: **every parallel backend beats single-threaded `verifyta`** at scale, and **correctness is invariant** (all configurations agree on the verdict). The headline is that **the adaptive Metal backend is the fastest configuration on every model**, beating the 12-thread OpenMP backend at every large N and reaching 3.4–7.3× over `verifyta`.

Naive metal: a win on expansion, a loss on merge. On Fischer, two-lock and arbiter the frontier is a large set of mostly-independent zones with small per-key antichains, so runtime is

Table 1: State-count parity vs UPPAAL `verifyta` at the largest timed size. On the first three models the counts match *exactly* (explored counts, omitted, match too); on the last two PTCTLMC's convex-union merge stores strictly fewer.

benchmark	N	PTCTLMC stored	UPPAAL stored	
Fischer	12	2 681 780	2 681 780	exact
two-lock	11	860 477	860 477	exact
arbiter	16	589 824	589 824	exact
CSMA/CD	14	618 496	991 232	1.6× fewer
philosophers	12	574 299	1 140 361	2.0× fewer

Table 2: Wall-clock seconds (min-of-k). `metal` is the naive one-thread-per-bucket merge; `metal-ad` is `-adaptive` (section 4.7). The fastest backend is in **bold**; the last column is its speed-up over `vfy`. `metal-ad` wins every row—and rescues the philosophers blow-up where the naive merge reaches 122 s.

model	N	stored	`seq`	`omp`	`metal`	`metal-ad`	`vfy`	best/`vfy`
Fischer	12	2 681 780	59.67	14.95	9.89	**8.25**	51.01	6.2×
two-lock	11	860 477	14.57	3.80	2.13	**1.80**	13.24	7.3×
arbiter	16	589 824	10.40	2.53	1.61	**1.57**	5.29	3.4×
CSMA/CD	14	618 496	11.94	3.32	2.98	**1.61**	9.70	6.0×
philosophers	11	188 687	4.29	1.03	7.85	**0.54**	2.41	4.5×
philosophers	12	574 299	20.74	5.17	121.93	**2.59**	10.88	4.2×

dominated by the uniform arithmetic of *expansion*—the GPU's strength—and even the naive `metal` beats OpenMP (Fischer/two-lock N=9–12; arbiter N=13–16). But on the merge-heavy models the naive one-thread-per-bucket merge (section 4) collapses: on philosophers the exact convex union fires constantly, and a handful of weak GPU lanes each grind through an $O(k^2 \dim^3)$ per-bucket load while the CPU idles. The effect is dramatic and super-linear—at philosophers N=12 the naive Metal merge takes **122 s**, an order of magnitude *slower* than plain `seq`. This is the empirical face of the tension in the abstract: expansion is parallel *across* states, but the merge is sequential *within* a bucket.

Adaptive metal fixes it. Routing the merge per level (section 4.7)—GPU when buckets are many and thin, the 12 CPU cores when they are few and fat—turns that loss into the best result on the board. Figure 10 plots the ratio: `-adaptive` is a 14× speed-up on philosophers N=11 and a 47× speed-up at N=12 (121.9 → 2.59 s), a 1.9× win on CSMA, and—crucially—never a regression on the already-fast expansion-dominated models (ratio ≈ 1 on Fischer, two-lock, arbiter, since the calibration simply leaves the merge on the GPU there). With adaptive on, `metal-ad` beats OpenMP at every large N across *all five* models (table 2).

Memory: the honest ceiling. Table 3 and fig. 11 report peak RSS. Ordering is consistent: `seq` $\lesssim$ `omp` < `metal` < `metal-ad`, because the GPU path double-buffers zones into GPU-facing buffers and the adaptive path adds host merge buffers on top. UPPAAL is by far the leanest (0.1–0.6 GB, an order of magnitude below any PTCTLMC backend)—its minimal-constraint federations trade time for space, the opposite of our trade. Memory, not time, is what bounds the largest instances on a 24 GB laptop: `metal-ad` reaches ∼13 GB at Fischer N=12, and one step further exhausts RAM for every backend. This is where a datacentre GPU (section 5) with far more memory changes the calculus.

A third negative result: frontier chunking does not buy memory. Since memory—not time—is the binding constraint, the obvious lever is to stop materialising a whole BFS level at once.

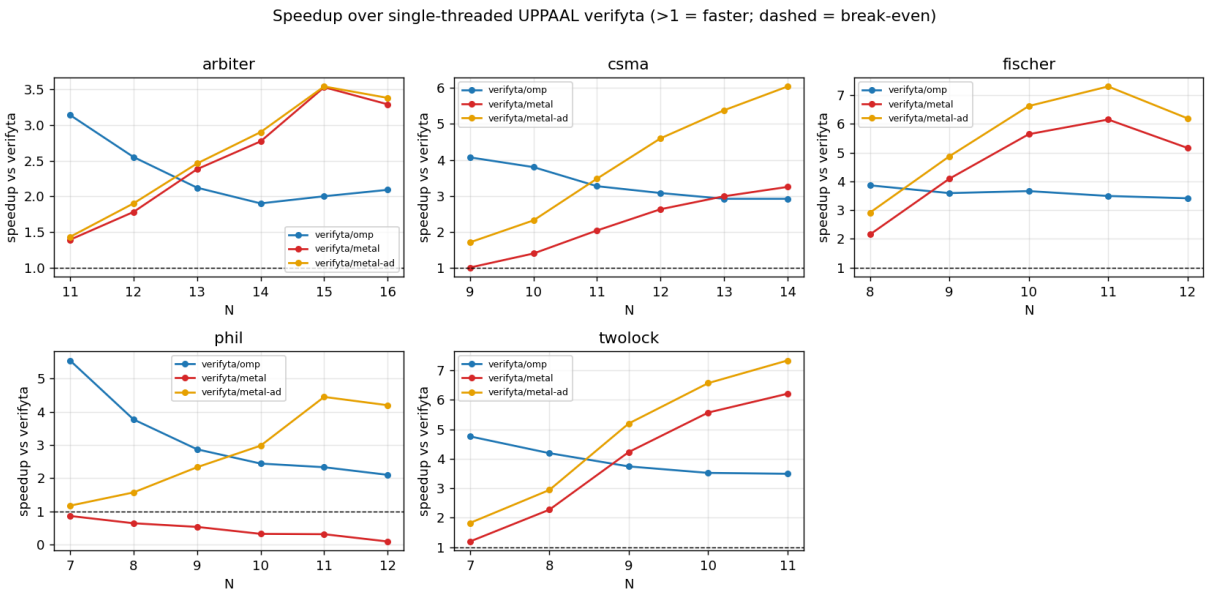


Figure 9: Speed-up over single-threaded UPPAAL `verifyta` versus model size N (a point above the dashed line is faster than `verifyta`). `omp` beats `verifyta` on every model; `metal-ad` (adaptive) is above it almost everywhere and is the top line at large N, including on philosophers where the *naive* `metal` (not shown here; see fig. 10) falls below break-even. (Inflated values at small N are UPPAAL's fixed ∼50 ms start-up, negligible at the sizes that matter—read each curve at its right-hand end.)

Table 3: Peak resident memory (GB) at the largest timed size. `metal-ad` is heaviest (GPU double-buffering + host merge buffers); `verifyta` is an order of magnitude leaner.

model	N	`seq`	`omp`	`metal`	`metal-ad`	`vfy`
Fischer	12	7.45	7.84	6.54	11.24	0.55
two-lock	11	1.92	2.33	3.43	4.04	0.18
arbiter	16	2.75	3.25	5.20	6.39	0.11
CSMA/CD	14	2.15	2.22	2.94	3.90	0.14
philosophers	12	1.35	1.82	2.31	2.72	0.19

We added `-chunking N`, which expands the frontier in bounded batches so the unified-memory pools are sized to a *chunk* rather than to the widest level (the frontier is a worklist, so batching only changes the schedule, never the verdict). It does exactly what it claims at the level we control: at Fischer N=11 it cuts the peak expand pool from 1.44M work items to 41.6k, and the summed buffer length from 2.15 GB to 0.26 GB—an 8× reduction.

And it does not reduce peak RSS at all (table 4). Worse, small chunks *increase* it while costing up to 12× in run time. The explanation is the same one that makes the memory gap hard to close: `vmmap` attributes the bulk of the footprint not to our buffers but to `IOAccelerator` regions the Metal driver holds *non-volatile* and reclaims only lazily. A smaller chunk means *more* levels, hence more per-level transient allocations for the driver to retain—so the lever we control (pool size) is not the quantity that sets the peak. We keep the flag, default off, and report it as a negative result: on this runtime, bounding your own allocations is not sufficient to bound resident memory.

7 Discussion and conclusion

Two backends, one control structure, one correctness oracle—and, crucially, one that we hold honest by matching UPPAAL's explored/stored counts *exactly* (table 1), so every speed-up compares like

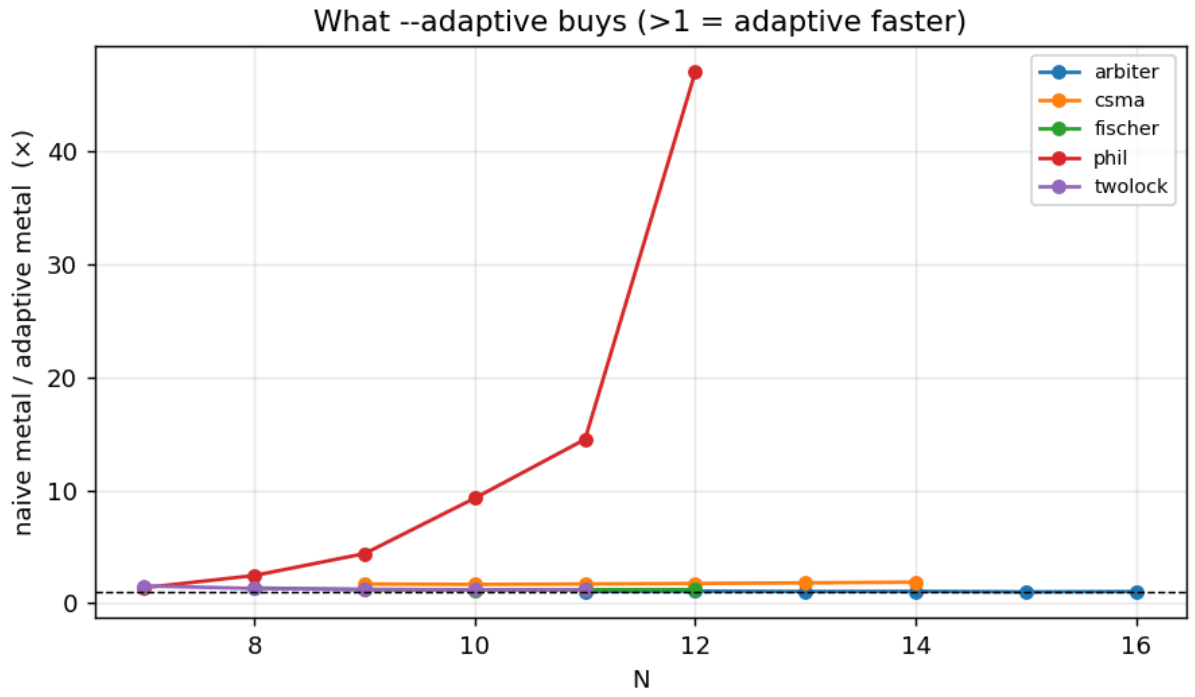


Figure 10: What `-adaptive` buys: naive-merge time / adaptive-merge time (>1 = adaptive faster). It is a 47× speed-up on the merge-heavy philosophers at N=12 and a modest win on CSMA, while staying ≈1 (no regression) on the expansion-dominated models—the calibration routes each level to the faster path.

Table 4: Frontier chunking at Fischer N=12: wall-clock seconds and peak RSS (GB) versus chunk size ("off" = `metal-ad`). Bounding the GPU pools leaves peak RSS unchanged while run time degrades sharply, because the resident peak is driver-retained memory, not our buffers.

chunk (frontier states)	off	256k	64k	16k	4k
wall-clock (s)	**8.25**	10.01	17.49	47.83	99.30
peak RSS (GB)	11.24	11.23	11.40	11.12	11.68

with like. The OpenMP backend earns its speed by *partitioning ownership* (shard-by-key, lock-free merge) and by *taming the allocator* (per-thread pools). The Metal backend earns its speed by keeping the GPU busy with the uniform work it likes (expansion) while a *parallel* host layer feeds it through unified memory with as few copies as possible—so the whole machine, GPU and all CPU cores, works at once. Every parallel backend beats a mature single-threaded tool on every model, and the adaptive Metal backend does so by 3.4–7.3× while being the fastest configuration throughout.

The decisive variable is the *ratio of expansion to merge*. Expansion is parallel *across* symbolic states—the GPU's home turf—but the antichain merge is sequential *within* a discrete-state bucket, and the GPU merge kernel spends one thread per bucket. When buckets are many and thin (Fischer, two-lock, arbiter) that is a fine fit and the GPU wins outright; when they are few and fat (philosophers) a few weak GPU lanes carry the whole load and the naive merge collapses—at N=12 it is 47× slower than routing that same merge onto the 12 CPU cores. Rather than guess which regime a level is in from static bucket statistics (which do not separate the cases), the `-adaptive` driver *measures* both paths' throughput and routes each level to the faster one; because it lives in the shared driver, the CUDA backend inherits it. That single mechanism turns the backend's worst case into its best result and leaves the good cases untouched. The remaining

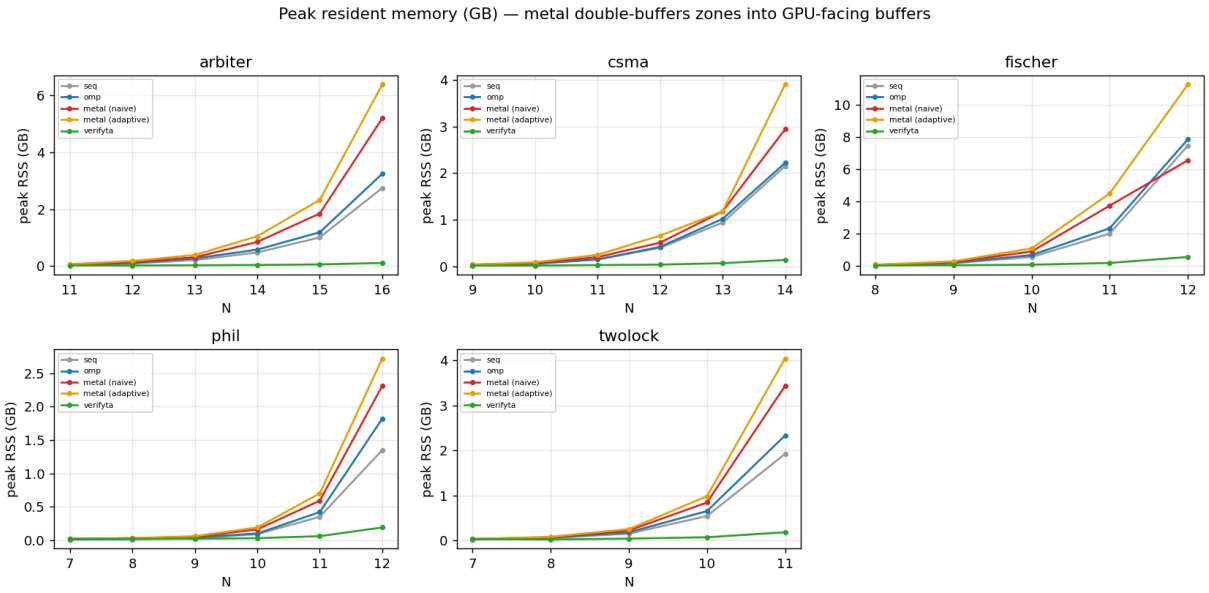


Figure 11: Peak RSS (GB) vs N per model. The GPU backends carry a memory premium from double-buffering into unified-memory buffers (adaptive most of all); `verifyta` is consistently the leanest. On a 24 GB machine this footprint, not run time, sets the ceiling.

honest limit is *memory*: the GPU backends pay a double-buffering premium (adaptive most of all), UPPAAL is an order of magnitude leaner, and on a 24 GB laptop peak RAM—not run time—is what caps the largest instances—and `-chunking`, which bounds the pools we allocate 8×, does not bound what the driver keeps resident, so it buys no memory and costs time. Two further attempts to cut the zone traffic (a GPU-resident arena and a direct frontier buffer) backfired for concrete, measurable reasons. A datacentre GPU (section 5)—far more memory and thousands more lanes—is where the same design has the most room to grow. The broader takeaways transfer well beyond this checker: profile relentlessly, suspect the memory plumbing before the compute, prefer ownership to locking, *match the parallel granularity to the data*—and when you cannot predict the right granularity, *measure it at run time* rather than hard-coding a heuristic.

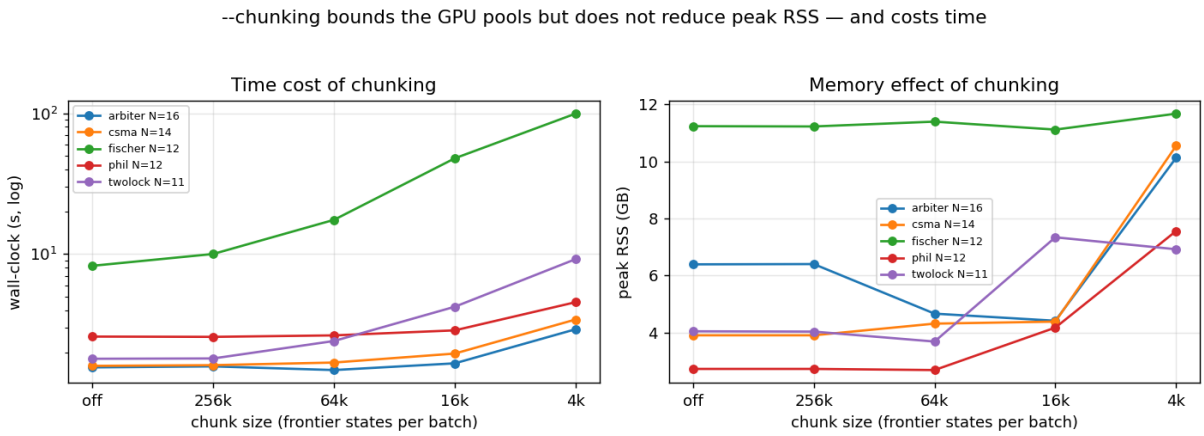


Figure 12: Frontier chunking across all five models (each at its largest N). Run time degrades monotonically as the chunk shrinks (left, log scale) while peak RSS stays flat or gets *worse* (right). The effect is not model-specific: bounding the pools *we* allocate does not bound what the GPU driver keeps resident.